> **Historical build:** this report records the Server-owned coordinator at commit da46e613b1. See the Proxy ownership refactor report for the current implementation and new-binary validation.

MILVUS NATIVE MULTISEARCH · MIXED-TRAFFIC EXPERIMENT · 1 NOVEMBER 2026

# Does MultiSearch compete fairly with an already saturated Search workload?

Continuous individual Search traffic saturates the server first. A second client then adds either ordinary Search or MultiSearch work, and is removed while the original client continues running.

> **Measured comparison against added ordinary Search**
>
> - At 160 additional logical searches/s, MultiSearch changes resident throughput retention by **+1.37 percentage points** and resident p99 by **-1.2%** versus the matched ordinary-Search injection.
> - At 480 additional logical searches/s, MultiSearch changes resident throughput retention by **+4.45 percentage points** and resident p99 by **-5.2%** versus the matched ordinary-Search injection.
> - At continuously backlogged secondary client, MultiSearch changes resident throughput retention by **+2.21 percentage points** and resident p99 by **-2.9%** versus the matched ordinary-Search injection.
>
> These compare API behavior under the same workload. They do not establish a general scheduler fairness guarantee or per-tenant isolation.

---

## 1. What is shared, and what the executor means

`multiSearch *proxy.MultiSearchExecutor` is a stateful dispatcher owned by the existing gRPC Server. It has no listener or independent service registration. The existing `MilvusService` receives the MultiSearch call on the same endpoint. Shared semaphores bound the active batches and child searches across all envelopes.

Every item re-enters the existing Search interceptor chain and calls the existing `Server.Search → Proxy.Search → DQL queue`. The coordination object is separate from leaf search execution; it does not provide a new search engine or reserved execution pool. In this prototype the gRPC Server owns this coordination state; it is not implemented as a `Proxy.MultiSearch()` method.

```
entry := proxyEntryInterceptor(limiter, getMetaCache)
search := func(ctx context.Context, req *milvuspb.SearchRequest) (*milvuspb.SearchResults, error) {
    // Fork mutable stats; the access-log interceptor similarly allocates
    // an independent object. Identity, deadlines and tracing are inherited.
    ctx = metrics.WithIndependentRPCStats(ctx)
    resp, err := entry(ctx, req, &grpc.UnaryServerInfo{Server: s, FullMethod:
milvuspb.MilvusService_Search_FullMethodName},
        func(ctx context.Context, req any) (any, error) {
            return s.Search(ctx, req.(*milvuspb.SearchRequest))
        })
    if err != nil {
        return nil, err
    }
```

```
    result, ok := resp.(*milvuspb.SearchResults)
    if !ok || result == nil {
        return &milvuspb.SearchResults{Status: merr.Status(merr.WrapErrServiceInternalMsg("Search entry returned an
invalid response"))}, nil
    }
    return result, nil
}
```

Proxy admission appends search tasks to the shared queue and removes them from its front. Ordinary search tasks inherit `BaseTask.IsSubTask() == false`; MultiSearch does not set the scheduler's special subtask flag. Thus the separate pool for scheduler-created subtasks is not selected merely because a search belongs to an envelope. QueryNode uses the configured FIFO policy with compatible-task merging. These paths contain no priority distinction based on whether the external call was Search or MultiSearch.

```
func (bt *BaseTask) IsSubTask() bool {
    return false
}
```

FIFO with task merging is not strict per-client fairness. Batch bursts, the wait-for-all barrier, admission limits, transport overhead, task merging and execution cost can all change observed shares. The experiment therefore uses a matched ordinary-Search injection control.

---

## 2. Workload and controls

| 5 s warmup | 15 s Search only | 30 s Search + injected traffic | 15 s Search only |
|---|---|---|---|

- **Resident client A:** 128 independent workers, each continuously calling ordinary Search and immediately issuing its next request when the prior one completes. A remains active through all phases; it is not a light probe.
- **Injected client B:** each job contains eight searches. The control issues eight concurrent Search RPCs and waits for all eight. MultiSearch sends one eight-item RPC, with concurrency eight. B uses a separate gRPC connection with the same test identity.
- **Fixed-rate conditions:** 160 and 480 additional logical searches/s (20 and 60 groups/s). Client group caps are 16 and 32 respectively, providing headroom while recording any rejected or late load.
- **Continuously backlogged condition:** B has 16 group workers, at most 128 logical searches per wave. The group barrier applies to both B variants. This is a demand-driven capacity-sharing test, not an equal offered-QPS test.
- **No-injection control:** A runs alone through the same three phase windows, to reveal time drift without B.
- **Ordering:** two valid repetitions with opposing API order. Fourteen valid formal runs follow a separate harness smoke check. The first 160/s pair was replaced after a binary-checksum command consumed CPU inside the test container during its MultiSearch phase (seconds 44–46). Both original runs remain in the excluded-data archive; both were rerun in Search-then-MultiSearch order after the main suite. No conclusion uses the contaminated pair.

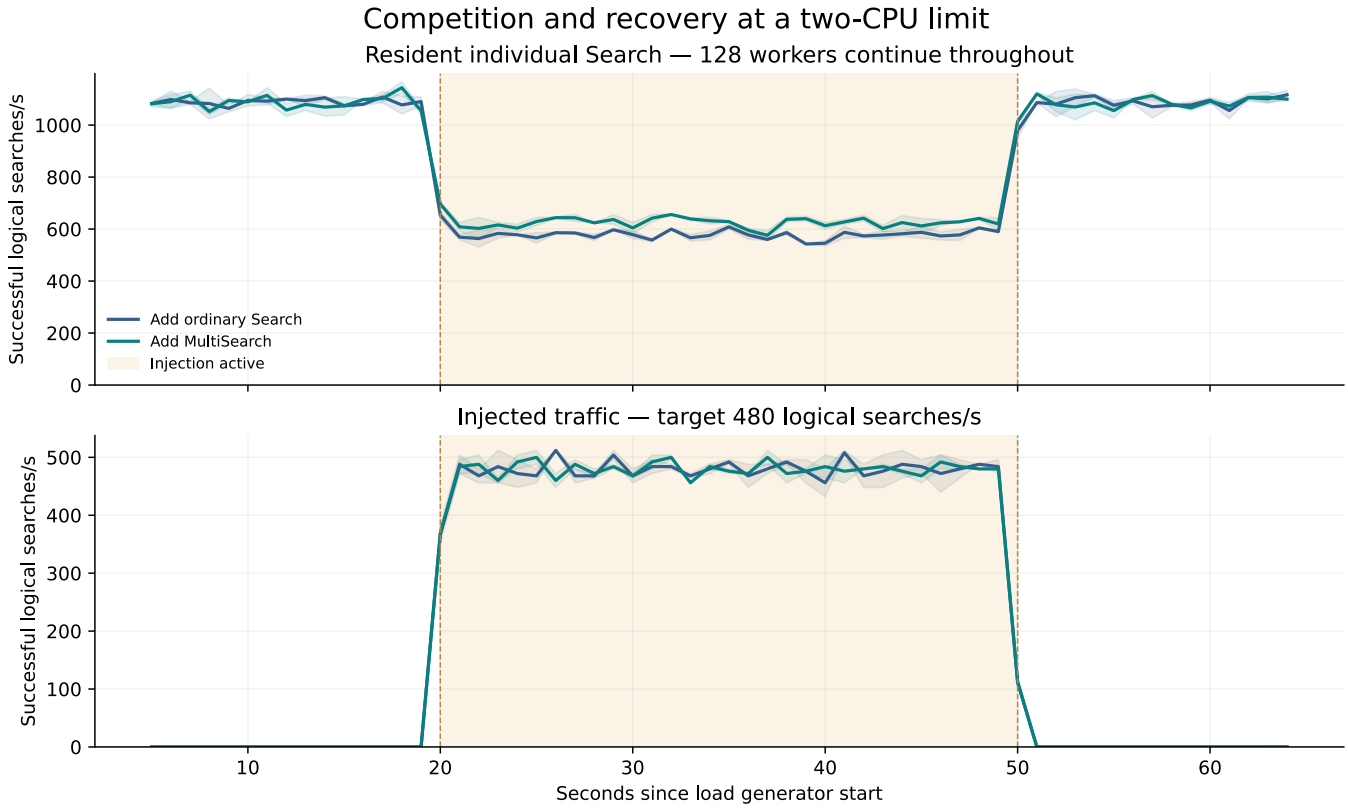
The unchanged prototype binary is based on v3.0.5. It uses a two-CPU quota/cpuset, GOMAXPROCS=4, and the existing 160,000-vector fixture: eight FLAT collections across two databases, nq=1, topK 5–8, dimensions 128/192/256 and L2/IP/COSINE. MultiSearch has 128 shared child permits and 32 admitted batches in this benchmark configuration; these exceed the API defaults of 64 and eight. There is no capacity reservation for A.

Each phase excludes its first three seconds from steady-state statistics. The resulting windows are 12 seconds of baseline, 27 seconds of mixed traffic, and 12 seconds of recovery. QPS counts successful completions in the window; latency includes calls that both start and complete inside it. A latency measures a single Search; B latency measures completion of all eight searches. Comparing A's p99 directly to B's p99 would not diagnose priority.

## 3. Impact on the existing Search client

| Injected B workload | A baseline QPS | A mixed QPS | A retained | A mixed p99, ms | A recovery QPS | CPU baseline → mixed |
|---|---|---|---|---|---|---|
| None | 1,077.3 | 1,082.6 | 100.5% | 177.19 | 1,088.0 | 96.9% → 96.9% |
| Search / 160/s | 1,089.0 | 922.7 | 84.7% | 198.32 | 1,079.8 | 97.1% → 97.0% |
| MultiSearch / 160/s | 1,088.5 | 937.2 | 86.1% | 196.01 | 1,090.0 | 96.8% → 97.2% |
| Search / 480/s | 1,088.7 | 578.4 | 53.1% | 286.16 | 1,090.5 | 97.2% → 97.0% |
| MultiSearch / 480/s | 1,085.9 | 625.3 | 57.6% | 271.23 | 1,087.1 | 96.9% → 97.3% |
| Search / backlogged | 1,092.0 | 575.2 | 52.7% | 284.57 | 1,095.5 | 97.1% → 96.8% |
| MultiSearch / backlogged | 1,094.7 | 600.8 | 54.9% | 276.38 | 1,091.6 | 97.3% → 96.9% |

Throughput values are means across repetitions. Retention is the mean of each run's mixed/baseline throughput ratio; p99 pools the raw latency samples across repetitions. The two-run retention ranges and all per-run measurements are in the summary JSON. No confidence intervals or statistical-significance claim is made.




480/s condition. Lines average the two aligned repetitions; shading spans the two observations for each second. Orange marks the injection interval. The resident client is never stopped or restarted between phases.

## 4. Does the added traffic get served?

| B workload | B successful logical/s | B group p99, ms | A + B logical/s | B share of completions | Minimum A completions in any measured second |
|---|---|---|---|---|---|
| Search / 160/s | 160.0 | 217.33 | 1,082.7 | 14.8% | 871 |
| MultiSearch / 160/s | 160.3 | 211.90 | 1,097.5 | 14.6% | 884 |
| Search / 480/s | 480.4 | 307.01 | 1,058.8 | 45.4% | 536 |
| MultiSearch / 480/s | 479.9 | 289.64 | 1,105.1 | 43.4% | 558 |
| Search / backlogged | 509.0 | 305.88 | 1,084.2 | 46.9% | 536 |
| MultiSearch / backlogged | 531.1 | 295.64 | 1,131.9 | 46.9% | 561 |

A 50/50 share is not the expected invariant for the backlogged condition: A independently replenishes each of 128 calls, while B replenishes groups only after their slowest search finishes. Some slots in a B group are idle while its remaining searches finish. The ordinary-Search B control has the same barrier, so differences between B variants are more informative than an assumed equal-share target. Client envelope concurrency is also not an exact measurement of active backend child work.

Across all 14 formal runs, 991,884 logical searches completed, including warmup and transitions. Recorded RPC/item errors: **0**; client admission drops: **0**; generator-late groups: **0**. The initial smoke check is excluded from these totals.

## 5. Interpretation and limits

**No additional harm to resident Search was observed from MultiSearch in these matched comparisons.** Injecting either API reduced resident throughput and raised its latency under CPU saturation. At each tested demand level, MultiSearch retained more resident throughput and had a lower resident p99 than the ordinary-Search injection. Both clients continued completing work, fixed-rate injection targets were met, and resident throughput recovered after injection stopped. This is consistent with ordinary competition for shared resources; it does not prove a scheduler fairness guarantee.

Use the fixed-rate pairs to determine whether MultiSearch imposes an extra cost on an already saturated ordinary-Search client at matched logical demand. Use the backlogged pairs to examine throughput sharing when both clients compete continuously. Recovery after B stops distinguishes temporary contention from lingering capacity loss. The no-injection control checks whether time drift alone could explain the changes.

The measurements and code-path audit support conclusions only for this same-identity, single-standalone, FLAT workload with the stated limits. CPU is measured for the entire standalone process, including QueryNode and background components. This does not isolate Proxy CPU or establish fairness across tenants, clusters, heterogeneous costs, much larger batches, different indexes, failures or long durations. The host is shared, affinity does not reserve sibling hyperthreads or NUMA placement, and no PSC/network fault is injected. Default MultiSearch admission settings can produce different backpressure.

No scheduler priority, server code or native binary was changed for this experiment. There is no inference that a dispatcher field constitutes an independent serving service. The new artifacts are the mixed-load driver, resource monitor, raw results and this report.

## 6. Reproduce and audit

```
go build -o bin/multisearch-mixed-load ./examples/multisearch/mixed_load
.venv/bin/python examples/multisearch/run_mixed.py
.venv/bin/python examples/multisearch/summarize_mixed.py
python examples/multisearch/report_mixed.py
```

The runner saves and restores the owned containers' CPU settings. It checks a short run for exactly 100 admitted eight-item groups and no RPC errors/drops before the formal suite. Samples include per-phase distributions, per-second completion counts, active client-call samples, CPU/memory/Go runtime and Proxy queue metrics. No production collections are used.

---